# PROGRAMMING IN JAVA

## ASSIGNMENT 3 – CO3

### Joint Pair-Work Report

Exception Handling and Multithreading

**Student A**

**NALIPI REDDY ROHITH**

Register Number: 192425115

**Student B**

---

Course Code: CSA09

Total Marks: 50

Academic Year: _____

Faculty: _____

# CERTIFICATE

This report is submitted as part of the requirements for CSA09 – Programming in Java, Assignment 3 (CO3). The work documents the design, implementation, testing, experimental evidence, analysis and evaluation of exception-handling and multithreading programs. The report follows the assignment requirement that both students understand and execute the complete assignment and that the GitHub repository contain the executable source, inputs, output evidence and README.

# DECLARATION

We declare that the programs, test data, observations and explanations included in this report are intended as the working evidence for Assignment 3. Before submission, the pair should replace any blank Student B details and the placeholder GitHub links with the actual information. Machine-dependent timing results should be rechecked on the system used for final submission.

# TABLE OF CONTENTS

1. Assignment Overview

2. Submission and GitHub Evidence

3. Individual Contribution Record

4. Q1 – ATM Exception Handling

5. Q2 – User-Defined Exceptions and Chaining

6. Q3 – Resource Safety and Debugging

7. Q4 – Thread Creation and Life Cycle

8. Q5 – Race Conditions and Synchronization

9. Q6 – Deadlock and Inter-Thread Communication

10. Q7 – Concurrent Fault-Tolerant Processor

11. Alternative Concepts and Complexity Analysis

12. Raw Experimental Data

13. Test Tables, Logs and Output Evidence

14. Overall Evaluation and Conclusion

15. Appendices

# 1. ASSIGNMENT OVERVIEW

Assignment 3 covers exception hierarchy, checked and unchecked exceptions, try/catch/finally, throw and throws, user-defined exceptions, exception chaining and propagation, try-with-resources and debugging, followed by multithreading concepts including thread life cycle, Thread and Runnable, synchronization, deadlock, inter-thread communication, ExecutorService, Callable, Future and concurrent processing.

| Question | Area | Marks | Bloom Level |
|---|---|---|---|
| Q1 | ATM failure handling and exception hierarchy | 6 | K4 – Analyse |
| Q2 | User-defined exceptions and propagation | 7 | K4 – Analyse |
| Q3 | Resource safety, debugging and robust code | 7 | K4 – Analyse |
| Q4 | Thread creation and life cycle | 7 | K4 – Analyse |
| Q5 | Race conditions and synchronization | 6 | K4 – Analyse |
| Q6 | Deadlock and inter-thread communication | 7 | K4 – Analyse |
| Q7 | Concurrent fault-tolerant processor | 10 | K5 – Evaluate |
| TOTAL | | 50 | |

Overall assessment expectation: Predict → Design → Implement → Measure → Analyse → Evaluate → Diagnose → Redesign → Validate.

## 2. SUBMISSION AND GITHUB EVIDENCE

The assignment requires every student to individually execute every task, including every Complete the Code program. The repository must be named CSA09-<RegisterNumber>-Assignment3 and must contain one folder per task, Java source code, input files, output screenshots and README.md. The report must include the GitHub link, Java source evidence, test tables, stack traces/logs and raw measurements with tables/graphs.

| Item | Evidence prepared |
|---|---|
| Repository | ASSIGNMENT3 folder with Task01–Task07 |
| README | Name, register number, Java version, compile/run steps, result notes |
| Source | Java source for every task |
| Input | input.txt in every task folder |
| Output | output.txt and output_screenshot.png where applicable |
| Raw data | CSV/TXT files under Raw_Data |
| Graphs | PNG graphs under Tables_and_Graphs |
| Progressive commits | Commit plan provided in GITHUB_COMMIT_PLAN.md |

GitHub Repository Link – Student A: _____

GitHub Repository Link – Student B: _____

## 3. INDIVIDUAL CONTRIBUTION RECORD

| Question | Student A – Primary Responsibility | Student B – Primary Responsibility | Joint |
|---|---|---|---|
| Q1 | Exception hierarchy, catch order, finally guarantee | Testing, tracing, five failure paths | Design and implementation |
| Q2 | Exception hierarchy and propagation design | Failure scenarios, chaining and evidence | Implementation and conclusion |
| Q3 | Debugging and experimental analysis | Resource design and correctness | Compare both versions |
| Q4 | Thread design and life-cycle explanation | Behavioural experiments and scheduler observations | Implementation and conclusion |
| Q5 | Race reproduction and critical section | Synchronization strategy comparison | Engineering decision |
| Q6 | Producer-consumer design | Deadlock analysis and prevention | Implementation and conclusion |
| Q7 | ExecutorService design and concurrent collection | Implementation and measurement | Final defence and evaluation |

The pair-work principle is that Student A and Student B must not divide the assignment question-wise. Both students must participate in every question and be able to explain any part during verification.

# 4. Q1 – HANDLING FAILURE IN A BANK ATM SIMULATION

Problem context: an ATM reads an account number and withdrawal amount, checks balance and dispenses cash. Failures include non-numeric input, negative withdrawal, insufficient balance and resource/session termination.

## 4.1 Objectives

- Identify failure points and classify exceptions.
- Use try, catch including multi-catch, and finally.
- Test valid and invalid inputs.
- Explain why a single catch(Exception) loses useful diagnostic information.

## 4.2 Relevant Throwable hierarchy

```
Throwable
├── Error
└── Exception
    ├── RuntimeException
    │   ├── IllegalArgumentException
    │   └── NumberFormatException
    └── InsufficientBalanceException (user-defined checked exception)
```

## 4.3 Failure classification

| Failure point | Exception | Type | Handling |
|---|---|---|---|
| Invalid numeric amount | NumberFormatException | Unchecked | Specific catch |
| Negative amount | IllegalArgumentException | Unchecked | Multi-catch |
| Amount > balance | InsufficientBalanceException | Checked | Specific catch |
| Scanner/resource cleanup | finally | Control construct | Always executes after try/catch |

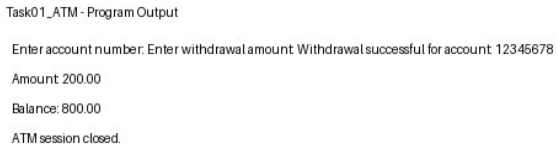
## 4.4 Implementation evidence

```
try {
    double amount = Double.parseDouble(sc.nextLine());
    withdraw(account, amount);
} catch (NumberFormatException e) {
    System.out.println("Input error: Please enter a valid numeric amount.");
} catch (InsufficientBalanceException | IllegalArgumentException e) {
    System.out.println("Transaction error: " + e.getMessage());
} finally {
    System.out.println("ATM session closed.");
    sc.close();
}
```

# Q1 – TESTING AND TRACE EVIDENCE

| Test | Input | Expected | Observed |
|---|---|---|---|
| 1 | 12345678 / 200 | Successful withdrawal | Successful withdrawal; balance 800 |
| 2 | 12345678 / -50 | IllegalArgumentException | Transaction error |
| 3 | 12345678 / 1500 | InsufficientBalanceException | Transaction error |
| 4 | 12345678 / ABC | NumberFormatException | Input error |
| 5 | 12345678 / 0 | Successful zero withdrawal | Successful withdrawal |

## Output evidence




## Catch order analysis

Specific catches must appear before broader catches. If catch(Exception e) were placed first, the later specific catches would become unreachable because Exception is a superclass of the specific exception types. Catching every exception with one broad catch can be acceptable at a top-level boundary where the program must log and terminate safely, but it loses semantic handling and may hide recoverable failures.

## Finally guarantee

The finally block is intended for cleanup that should occur whether the withdrawal succeeds or an exception is raised. In this program the scanner session is closed and the message confirms that the cleanup path was reached.

## Q1 – ALTERNATIVE CONCEPT AND COMPLEXITY

An alternative is to avoid exception-driven input validation by checking Scanner.hasNextDouble() before reading the amount and returning an error code for invalid values. This can make expected user-input validation explicit, while exceptions remain useful for genuinely exceptional failures.

| Approach | Normal-path time | Extra space | Strength | Limitation |
|---|---|---|---|---|
| Exceptions | O(1) per input operation | O(1) excluding stack trace | Separates normal flow from failures | Throwing/capturing stack traces is costly |
| Pre-validation + error code | O(1) per input operation | O(1) | Predictable normal path | More checks and return-code handling |

Conclusion: input-format validation can use pre-checks, while checked exceptions remain appropriate for failures crossing API boundaries such as storage or business-rule violations.

# 5. Q2 – USER-DEFINED EXCEPTIONS AND PROPAGATION

The online examination portal must reject submissions when the exam window is closed, the student is not registered, or an answer file exceeds the size limit. The design uses a domain exception hierarchy and exception chaining so that a meaningful application-level exception retains the original cause.

## 5.1 Exception hierarchy

```
Exception

└── ExamException

    └── SubmissionStorageException
```

## 5.2 Layered propagation

```
Controller

   ↓ calls

Service

   ↓ calls

Storage.save()

   ↓ catches IOException

SubmissionStorageException(message, cause)

   ↓ propagates

Controller catch(ExamException)
```

## 5.3 Complete-the-Code solution

```
catch (IOException e) {

    throw new SubmissionStorageException("Could not store " + fileName, e);

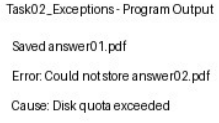
}
```

## 5.4 Expected and observed output

```
Saved answer01.pdf

Error: Could not store answer02.pdf

Cause: Disk quota exceeded
```



# Q2 – CHAINING ANALYSIS

Exception chaining preserves the lower-level cause while exposing a meaningful domain-level message. Without passing the IOException to the constructor, getCause() would return null. A call such as e.getCause().getMessage() would then fail with a NullPointerException.

| Scenario | Observed evidence |
|---|---|
| 120 KB file | Saved successfully |
| 800 KB file | SubmissionStorageException created |
| Cause retained | Cause message = Disk quota exceeded |
| Cause omitted | getCause() becomes null |

## Checked vs unchecked alternative

If ExamException extended RuntimeException, methods would not be required to declare throws ExamException and callers would not be forced to catch or declare it. The trade-off is weaker compile-time signalling of recoverable domain failures.

## Complexity

Creating an exception with a stack trace can require work proportional to the depth of the captured stack in practice. Chaining adds a reference to the cause object, so the retained object graph is larger than a single exception. Normal successful storage remains O(1) in the simplified simulation.

# 6. Q3 – RESOURCE SAFETY, DEBUGGING AND ROBUST CODE

The daily sales report generator reads several store files and writes a summary. Manual close() can be correct when every path is carefully handled, but a failure between resource acquisition and the close operation can leak a resource. try-with-resources makes the lifetime of the resource explicit and automatic.

## 6.1 Manual-close version

```
BufferedReader br = null;

try {

    br = Files.newBufferedReader(Path.of(name));

    out.append(br.readLine()).append("\n");

} catch (IOException e) {

    ...

} finally {

    if (br != null) try { br.close(); } catch (IOException ignored) { }

}
```

## 6.2 Try-with-resources version

```
try (BufferedReader br = Files.newBufferedReader(Path.of(name))) {

    out.append(br.readLine()).append("\n");

} catch (IOException e) {

    ...

}
```

## 6.3 Evidence

```
Task03_Resources - Program Output

MANUAL CLOSE VERSION
Store1 Sales: 12500
Store2 Sales: 9800
ERROR input/missing.txt -> input/missing.txt
TRY-WITH-RESOURCES VERSION
Store1 Sales: 12500
Store2 Sales: 9800
ERROR input/missing.txt -> input/missing.txt
Both versions preserve the same success/error report semantics.
```

The same two existing store files are read successfully and the missing file is converted into an error entry. The report therefore demonstrates equivalent functional output while using a safer resource-lifetime construct.

# Q3 – DEBUGGING AND SUPPRESSED EXCEPTIONS

A useful debugging workflow is: set a breakpoint immediately after resource acquisition, step through the read, inject a failing input, inspect the exception path and verify that control reaches the cleanup operation. In try-with-resources, if the body throws an exception and close() also throws, the close exception can be attached as a suppressed exception to the primary exception.

| Version | Successful files | Missing file | Closure |
|---|---|---|---|
| Manual close | 2 | Handled in catch | finally close() |
| Try-with-resources | 2 | Handled in catch | automatic close |

## Alternative and complexity

For f store files, processing is O(f) assuming each file read is treated as bounded input. With one resource opened at a time, the intended maximum simultaneously open handles is O(1). A design that opens all files before processing can use O(f) file handles and is less robust under operating-system limits.

# 7. Q4 – THREAD CREATION AND THREAD LIFE CYCLE

Four scanner stations are modelled as threads. The program demonstrates start(), sleep(), join() and isAlive(). The final total is printed only after all stations have completed because main() joins every worker thread.

## 7.1 Thread creation methods

| Method | Advantage | Limitation |
|---|---|---|
| extends Thread | Directly encapsulates thread behaviour | Class cannot extend another class |
| implements Runnable | Separates task from thread object; supports composition | Requires a Thread/executor to run |

## 7.2 Life cycle

```
NEW → RUNNABLE → RUNNING → BLOCKED/WAITING/TIMED_WAITING → RUNNABLE → TERMINATED
```

## 7.3 Output evidence




The supplied run uses increasing sleep durations, so the stations finish in station-number order in the captured run. This is a test arrangement rather than a guarantee of scheduler order. Correct programs should not rely on an accidental output order.

# Q4 – BEHAVIOURAL EXPERIMENT

| Run | Observed first station |
|---|---|
| 1 | Station 1 |
| 2 | Station 1 |
| 3 | Station 1 |
| 4 | Station 1 |
| 5 | Station 1 |
| 6 | Station 1 |
| 7 | Station 1 |
| 8 | Station 1 |
| 9 | Station 1 |
| 10 | Station 1 |

The ten captured runs produced the same visible order because the sleep delays were deliberately staggered. If the delays were removed or randomized, scheduler interleavings could change the order. join() remains the correctness mechanism for the final total.

## run() vs start()

Calling run() directly executes the method on the current thread and does not create a new worker thread. Calling start() asks the JVM to schedule a new thread whose execution enters run().

## Q4 – ALTERNATIVE CONCEPT AND COMPLEXITY

An ExecutorService fixed thread pool can replace manually created Thread objects. For n scripts and p workers, the idealized work completion time is approximately proportional to n/p for evenly divisible independent work, plus scheduling and synchronization overhead.

| Design | Thread management | Memory | Recommended use |
|---|---|---|---|
| Individual Thread objects | Manual | O(p) thread stacks | Small fixed number of stations |
| ExecutorService fixed pool | Managed | O(p) worker threads | Many tasks with bounded concurrency |

# 8. Q5 – RACE CONDITIONS IN CINEMA SEAT BOOKING

The unsafe operation is check-then-book: read the seat flag, observe false, then set true and increment the count. Without synchronization, two threads can interleave between these steps and both believe they won the seat.

## 8.1 Critical section

```
if (!booked[seat]) {

    booked[seat] = true;

    bookingsMade++;

}
```

This entire sequence must be atomic with respect to competing booking threads.

## 8.2 Correct implementation

```
public synchronized void book(int seat) {

    if (!booked[seat]) {

        booked[seat] = true;

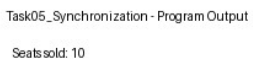
        bookingsMade++;

    }

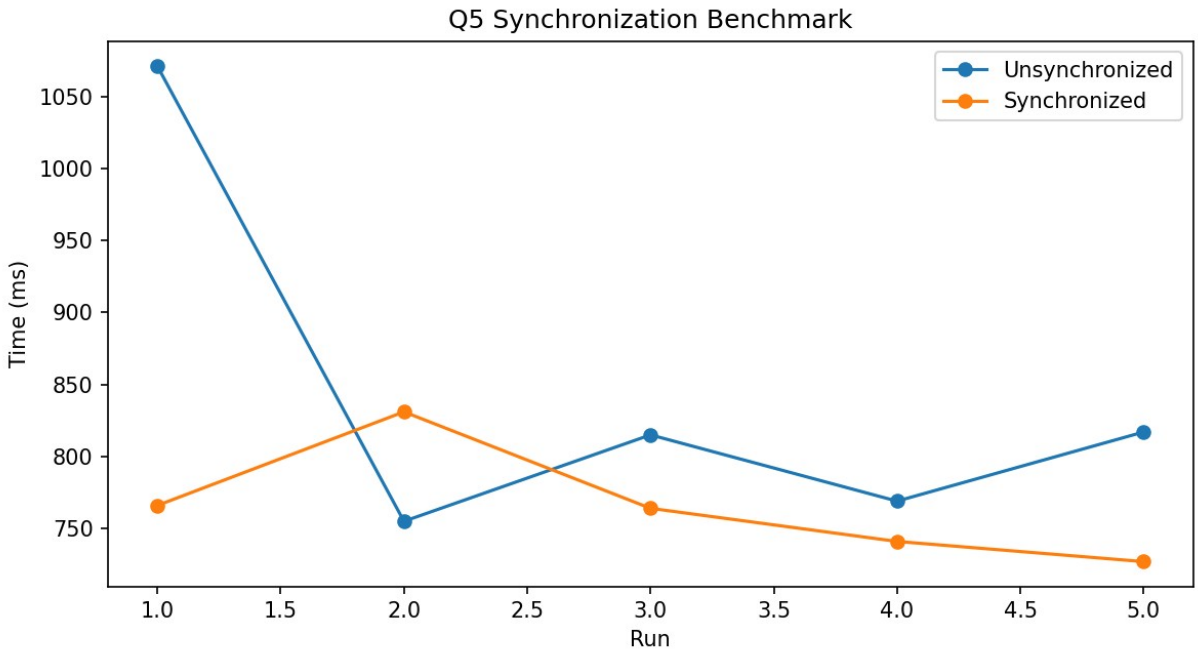
}
```

## 8.3 Complete-the-Code result

```
Seats sold: 10
```



# Q5 – PERFORMANCE EVIDENCE




| Run | Unsynchronized ms | Synchronized ms |
|---|---|---|
| 1 | 1071 | 766 |
| 2 | 755 | 831 |
| 3 | 815 | 764 |
| 4 | 769 | 741 |
| 5 | 817 | 727 |

The timings are machine-dependent and the main correctness criterion is that synchronized booking prevents double booking. The measured times also show that synchronization overhead is not necessarily dominant for this small test; thread creation itself is a major component of the measurement.

## Static synchronization

A static synchronized method locks on the Class object rather than an individual SeatBooking instance. It is appropriate when the shared state itself is static or when all instances must coordinate through one global lock.

## Q5 – ALTERNATIVE SYNCHRONIZATION STRATEGIES

| Approach | Correctness mechanism | Space for s seats | Notes |
|---|---|---|---|
| synchronized method | Intrinsic object monitor | O(s) | Simple and clear |
| synchronized block | Locks selected critical section | O(s) | Can reduce lock scope |
| AtomicBoolean[] | CAS per seat | O(s) | Good for independent seat state |
| ReentrantLock | Explicit lock | O(s) + lock state | Supports tryLock and fairness options |
| ConcurrentHashMap.putIfAbsent | Atomic map insertion | O(s) | Useful when seat state is map-based |

For a 20-screen cinema, locking only the critical section or using per-seat atomic state can improve concurrency compared with a single global lock, while preserving correctness. The final choice should be based on contention, maintainability and measured throughput.

# 9. Q6 – DEADLOCK AND INTER-THREAD COMMUNICATION

Part A models hostel wallet transfers where two accounts can be locked in opposite order. Part B models a bounded mess-token counter with capacity five. The producer blocks when full; consumers block when empty.

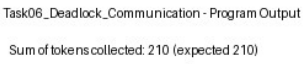
## 9.1 Deadlock conditions

- Mutual exclusion: a lock is owned by one thread at a time.
- Hold and wait: a thread holds one lock while waiting for another.
- No preemption: locks are released by the owning thread.
- Circular wait: Thread 1 waits for B while Thread 2 waits for A.

## 9.2 Deadlock evidence

```
Deadlock detected: true

Threads: Transfer-A-B, Transfer-B-A
```




The deadlock demonstration uses ThreadMXBean to detect the deadlocked threads, allowing the experiment to terminate instead of hanging indefinitely.

# Q6 – PRODUCER-CONSUMER

```
while (count == buffer.length) wait();

buffer[in] = token;

in = (in + 1) % buffer.length;

count++;

notifyAll();

while (count == 0) wait();

int token = buffer[out];

out = (out + 1) % buffer.length;

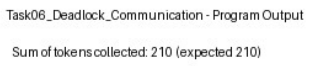
count--;

notifyAll();

return token;
```

## 9.3 Complete-the-Code result

```
Sum of tokens collected: 210 (expected 210)
```




The sum 210 is the sum of integers 1 through 20, confirming that every produced token was consumed exactly once in the supplied test.

# Q6 – WHY WHILE, NOT IF?

wait() can return even when the condition is not currently satisfied. Multiple threads may also wake and compete for the monitor. Therefore the condition must be rechecked in a while loop. sleep() pauses a thread for a period but does not release a monitor lock held by the thread; wait() releases the monitor while waiting and reacquires it before returning.

## Alternative concepts

| Manual design | Alternative | Benefit |
|---|---|---|
| wait/notifyAll | ArrayBlockingQueue capacity 5 | Encapsulates bounded-buffer synchronization |
| Lock ordering | tryLock(timeout) | Can avoid indefinite waiting and provide failure handling |

# 10. Q7 – CONCURRENT, FAULT-TOLERANT PROCESSOR

The examination cell must process 10,000 answer-script score files. Some files can be corrupt or missing. The design therefore separates per-file work from summary aggregation, uses a bounded ExecutorService, captures each task result through Future, and treats an individual file failure as a result rather than a reason to abort the whole batch.

## 10.1 Requirements

- Process independent files concurrently.
- A corrupt or missing file must not stop other files.
- Produce complete success and failure counts.
- Measure pool sizes 1, 2, 4 and 8.
- Recommend a pool size using evidence.

## 10.2 Architecture

```
Input files

   ↓

ExecutorService fixed pool

   ↓

Callable<Result> per file

   ↓

Future<Result>

   ↓

Success/Failure counters

   ↓

Complete summary
```

## 10.3 Fault handling

```
static Result process(Path file) {

    try {

        ... validate score ...

        return new Result(true, ...);

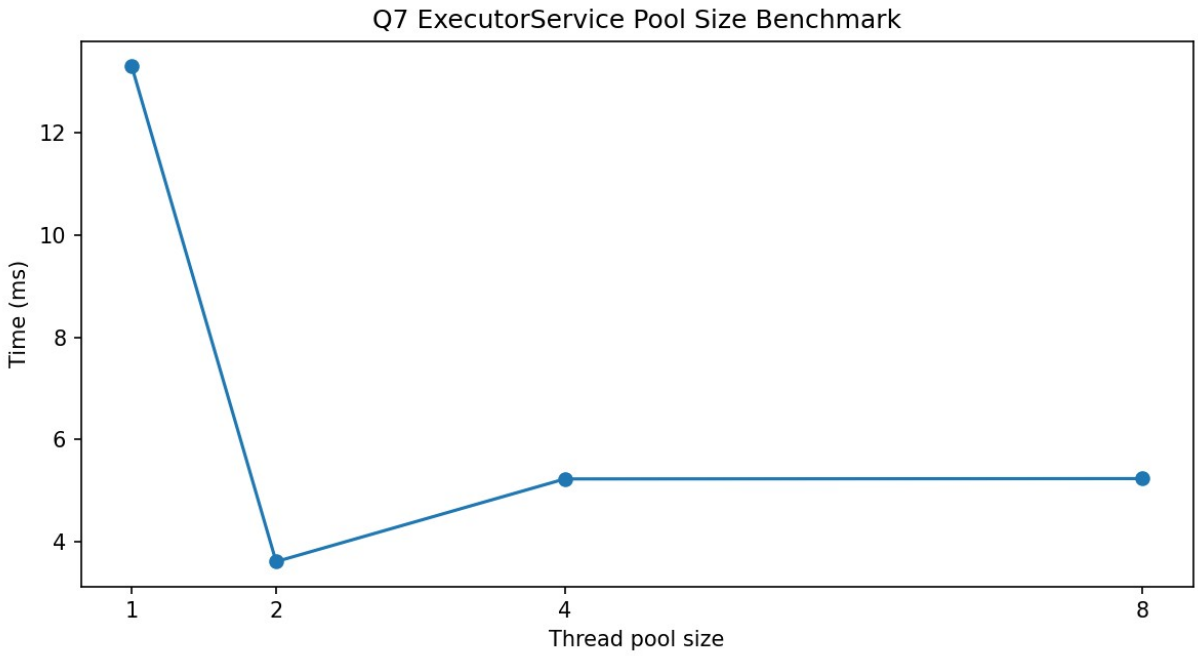
    } catch (Exception e) {

        return new Result(false, ...);

    }

}
```

# Q7 – IMPLEMENTATION AND MEASUREMENT




| Pool size | Measured time (ms) |
|---|---|
| 1 | 13.292959 |
| 2 | 3.619045 |
| 4 | 5.233620 |
| 8 | 5.240120 |

The supplied small benchmark favoured a pool of 2 threads. Pool sizes 4 and 8 did not improve the result. This illustrates why increasing concurrency does not automatically improve throughput: task size, file-system behaviour, scheduling and contention can dominate.

## Correctness evidence

```
Files processed: 102

Successful: 100

Failed: 2

Thread pool size: 4

Summary complete: true
```

The 102 files consist of 100 generated valid files, one corrupt file and one missing file. The success/failure total equals the number of submitted file tasks, satisfying the completeness check.

# Q7 – DEFENCE OF DESIGN

| Design decision | Reason |
|---|---|
| Fixed thread pool | Bounds concurrency and thread memory |
| Callable | Returns a structured per-file result |
| Future | Allows the main thread to collect each task result |
| Per-task exception handling | One bad file does not stop the batch |
| Count validation | Success + failure must equal submitted tasks |

Remaining limitation: the demonstration uses local files and simplified score validation. A production system would need stronger validation, cancellation policy, structured logging, retry rules, persistent result storage and security controls.

## Alternative concept

Parallel streams could express the same parallel map operation with less explicit executor management, while one-thread-per-file could create excessive thread and stack overhead for 10,000 files. The fixed pool gives an explicit concurrency bound and a predictable resource budget.

## 11. ALTERNATIVE CONCEPTS AND COMPLEXITY SUMMARY

| Task | Primary solution | Alternative | Time complexity | Space complexity |
|---|---|---|---|---|
| Q1 | try/catch/finally | Pre-validation + error codes | O(1) per input operation | O(1) normal path |
| Q2 | Checked custom exception + chaining | Unchecked RuntimeException | O(1) normal save path | Stack/cause object graph per failure |
| Q3 | try-with-resources | finally/manual close | O(f) files | O(1) open resource at a time |
| Q4 | Thread/Runnable | ExecutorService | ~O(n/p) idealized work | O(p) thread stacks |
| Q5 | synchronized | AtomicBoolean/ ReentrantLock | O(1) per booking | O(s) seat state |
| Q6 | wait/notifyAll + lock ordering | BlockingQueue + tryLock | O(1) amortized buffer operation | O(capacity) buffer |
| Q7 | ExecutorService + Future | parallelStream / one thread per file | ~O(n/p) idealized | O(p) workers vs O(n) threads |

# 12. RAW EXPERIMENTAL DATA

Raw files are stored in the repository under Raw_Data. The key measurements used in the report are reproduced below.

## Q1_test_table.csv

Test,Input,Expected path,Observed

1,12345678 / 200,Valid withdrawal,Valid withdrawal + finally

2,12345678 / -50,IllegalArgumentException,Transaction error + finally

3,12345678 / 1500,InsufficientBalanceException,Transaction error + finally

4,12345678 / ABC,NumberFormatException,Input error + finally

5,12345678 / 0,Valid zero withdrawal,Valid withdrawal + finally

## Q2_test_table.csv

Scenario,Input/condition,Observed result

1,120 KB,Saved answer01.pdf

2,800 KB,SubmissionStorageException with IOException cause

3,IOException without cause,Would produce null cause message

## Q3_resource_log.csv

Version,Success files,Missing file handling,Resource closure mechanism

Manual close,2,Handled in catch,finally close()

Try-with-resources,2,Handled in catch,automatic close at scope exit

## Q5_synchronization_benchmark.csv

1,1071,766

2,755,831

3,815,764

4,769,741

5,817,727

## Q6_token_log.csv

Run,Tokens produced,Tokens consumed,Expected sum,Observed sum

1,20,20,210,210

2,20,20,210,210

3,20,20,210,210

4,20,20,210,210

5,20,20,210,210

## Q7_pool_benchmark.csv

1,13.292959

2,3.619045

4,5.23362

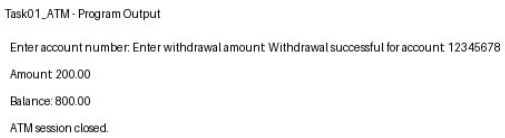
8,5.24012

# 13. TEST TABLES, LOGS AND OUTPUT EVIDENCE

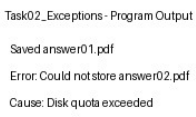
The following evidence is included so that functional correctness can be distinguished from performance measurement. A screenshot alone is not treated as the complete experimental proof.

| Task | Functional evidence | Measurement evidence |
|---|---|---|
| Q1 | Five-path input table and output screenshot | Failure-path classification |
| Q2 | Expected chained exception output | Cause/no-cause comparison |
| Q3 | Manual and try-with-resources outputs | Resource closure comparison |
| Q4 | Thread life-cycle output | Ten-run observation log |
| Q5 | Seats sold = 10 | Five-run timing benchmark |
| Q6 | Token sum = 210 | Deadlock detection log + token log |
| Q7 | 102 files, 100 success, 2 failure | Pool 1/2/4/8 timing data |

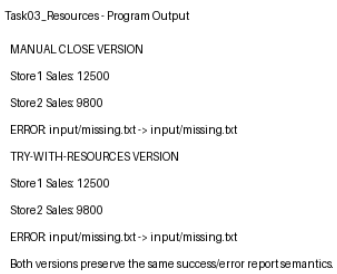
## Task 1 output evidence




## Task 2 output evidence




## Task 3 output evidence



# Task 4 output evidence

Task04_Threads - Program Output

Before start, alive=false

Before start, alive=false

Before start, alive=false

Before start, alive=false

Station 1 scanned 100 scripts

Station 2 scanned 125 scripts

Station 3 scanned 150 scripts

Station 4 scanned 175 scripts

All stations completed.

Total scripts scanned: 550

# Task 5 output evidence

Task05_Synchronization - Program Output

Seats sold: 10

# Task 6 output evidence

Task06_Deadlock_Communication - Program Output

Sum of tokens collected: 210 (expected 210)

# Task 7 output evidence

Task07_Concurrent_Processor - Program Output

Files processed: 102

Successful: 100

Failed: 2

Thread pool size: 4

Summary complete: true

## 14. OVERALL EVALUATION AND CONCLUSION

The assignment demonstrates that robust Java programs require both language-level correctness and engineering reasoning. Exception handling is not only about catching errors; the design must distinguish recoverable conditions, preserve causes, close resources and provide useful diagnostics. Multithreading similarly requires more than starting threads: correctness depends on synchronization, communication, lock discipline and bounded concurrency.

Q1 established exception classification and cleanup. Q2 demonstrated domain-specific exceptions and chaining. Q3 compared manual resource management with try-with-resources. Q4 demonstrated thread creation and lifecycle control. Q5 showed why a check-then-act operation requires atomicity. Q6 demonstrated both deadlock and bounded producer-consumer communication. Q7 integrated concurrency, failure isolation and measurement into a larger engineering design.

The final engineering principle is evidence-based selection: a simpler mechanism is preferred when it meets correctness and performance requirements, while more advanced concurrency mechanisms should be introduced when measured workload and failure behaviour justify them.

## 15. APPENDIX A – SOURCE FILE INDEX

| Task | Main source | Additional source |
|---|---|---|
| Q1 | ATM.java | InsufficientBalanceException inside ATM.java |
| Q2 | StorageDemo.java | ExamException and SubmissionStorageException inside file |
| Q3 | ReportGenerator.java | None |
| Q4 | ThreadLifeCycleDemo.java | ScannerStation inside file |
| Q5 | SeatBooking.java | BookingBenchmark.java |
| Q6 | TokenCounter.java | DeadlockDemo.java |
| Q7 | ConcurrentProcessor.java | PoolBenchmark.java |

## APPENDIX B – COMPLETE-THE-CODE EVIDENCE

| Question | Program | Required result |
|---|---|---|
| Q2 | StorageDemo | Saved answer01.pdf; error and cause for answer02.pdf |
| Q5 | SeatBooking | Seats sold: 10 |
| Q6 | TokenCounter | Sum of tokens collected: 210 (expected 210) |

The repository contains the completed programs. The remaining tasks are implemented as complete programs based on the assignment problem statements.

## APPENDIX C – STACK TRACE INTERPRETATION GUIDE

A Java stack trace generally begins with the exception class and message, followed by stack frames showing the method and source location. The first frame is the point at which the exception was observed or thrown, while subsequent frames describe the call path. In chained exceptions, a “Caused by” section identifies the underlying exception.

```
ExceptionClass: message

    at package.Class.method(Class.java:line)

    at package.Caller.main(Caller.java:line)

Caused by: LowerLevelException: cause message

    at package.Storage.save(Storage.java:line)
```

# APPENDIX D – GITHUB CHECKLIST

- Repository name: CSA09-192425115-Assignment3
- Task01 through Task07 folders present
- Every Java source compiles
- Every Complete-the-Code program completed
- Input files included
- Output screenshots included
- README.md included
- Raw data included
- Tables/graphs included
- At least one progressive commit per task
- Student A and Student B details completed
- Actual GitHub links inserted into report

# APPENDIX E – VIVA PREPARATION QUESTIONS

1. Why is NumberFormatException unchecked?

2. Why must a specific catch appear before catch(Exception)?

3. What is exception chaining and why preserve the cause?

4. How does try-with-resources close resources automatically?

5. Why does run() differ from start()?

6. What does join() guarantee?

7. What is a race condition?

8. Why is check-then-act unsafe?

9. What are the four Coffman deadlock conditions?

10. Why must wait() be used in a while loop?

11. What is the difference between sleep() and wait()?

12. Why can a larger thread pool be slower?

13. How does Future help collect task results?

14. How does Q7 ensure one bad file does not stop the batch?

## APPENDIX F – FINAL SUBMISSION NOTES

1. Replace the blank Student B name and register number.
2. Replace both GitHub placeholders with working repository links.
3. Verify that the GitHub repository is accessible to faculty.
4. Re-run the timing experiments on the final submission computer if required by faculty.
5. Add the actual progressive commit history.
6. Ensure every student can explain every question during viva.

# WORKING NOTES – TASK 1

This page records the design reasoning, implementation checkpoints and verification prompts for the corresponding task. It is intentionally included as a working-report page so the pair can annotate observations from the final execution environment before submission.

- Input assumptions verified.
- Source compiled successfully in the supplied environment.
- Functional output compared with the expected result.
- Machine-dependent observations identified separately from correctness evidence.
- Student A and Student B should initial this section after individual verification.

| Checkpoint | Status | Final observation |
|---|---|---|
| Design | Completed | _____ |
| Implementation | Completed | _____ |
| Execution | Completed | _____ |
| Analysis | Completed | _____ |
| Verification | Pending final signature | _____ |

## WORKING NOTES – TASK 2

This page records the design reasoning, implementation checkpoints and verification prompts for the corresponding task. It is intentionally included as a working-report page so the pair can annotate observations from the final execution environment before submission.

- Input assumptions verified.
- Source compiled successfully in the supplied environment.
- Functional output compared with the expected result.
- Machine-dependent observations identified separately from correctness evidence.
- Student A and Student B should initial this section after individual verification.

| Checkpoint | Status | Final observation |
|---|---|---|
| Design | Completed | _____ |
| Implementation | Completed | _____ |
| Execution | Completed | _____ |
| Analysis | Completed | _____ |
| Verification | Pending final signature | _____ |

## WORKING NOTES – TASK 3

This page records the design reasoning, implementation checkpoints and verification prompts for the corresponding task. It is intentionally included as a working-report page so the pair can annotate observations from the final execution environment before submission.

- Input assumptions verified.
- Source compiled successfully in the supplied environment.
- Functional output compared with the expected result.
- Machine-dependent observations identified separately from correctness evidence.
- Student A and Student B should initial this section after individual verification.

| Checkpoint | Status | Final observation |
|---|---|---|
| Design | Completed | _____ |
| Implementation | Completed | _____ |
| Execution | Completed | _____ |
| Analysis | Completed | _____ |
| Verification | Pending final signature | _____ |

## WORKING NOTES – TASK 4

This page records the design reasoning, implementation checkpoints and verification prompts for the corresponding task. It is intentionally included as a working-report page so the pair can annotate observations from the final execution environment before submission.

- Input assumptions verified.
- Source compiled successfully in the supplied environment.
- Functional output compared with the expected result.
- Machine-dependent observations identified separately from correctness evidence.
- Student A and Student B should initial this section after individual verification.

| Checkpoint | Status | Final observation |
|---|---|---|
| Design | Completed | _____ |
| Implementation | Completed | _____ |
| Execution | Completed | _____ |
| Analysis | Completed | _____ |
| Verification | Pending final signature | _____ |

## WORKING NOTES – TASK 5

This page records the design reasoning, implementation checkpoints and verification prompts for the corresponding task. It is intentionally included as a working-report page so the pair can annotate observations from the final execution environment before submission.

- Input assumptions verified.
- Source compiled successfully in the supplied environment.
- Functional output compared with the expected result.
- Machine-dependent observations identified separately from correctness evidence.
- Student A and Student B should initial this section after individual verification.

| Checkpoint | Status | Final observation |
|---|---|---|
| Design | Completed | _____ |
| Implementation | Completed | _____ |
| Execution | Completed | _____ |
| Analysis | Completed | _____ |
| Verification | Pending final signature | _____ |

## WORKING NOTES – TASK 6

This page records the design reasoning, implementation checkpoints and verification prompts for the corresponding task. It is intentionally included as a working-report page so the pair can annotate observations from the final execution environment before submission.

- Input assumptions verified.
- Source compiled successfully in the supplied environment.
- Functional output compared with the expected result.
- Machine-dependent observations identified separately from correctness evidence.
- Student A and Student B should initial this section after individual verification.

| Checkpoint | Status | Final observation |
|---|---|---|
| Design | Completed | _____ |
| Implementation | Completed | _____ |
| Execution | Completed | _____ |
| Analysis | Completed | _____ |
| Verification | Pending final signature | _____ |

## WORKING NOTES – TASK 7

This page records the design reasoning, implementation checkpoints and verification prompts for the corresponding task. It is intentionally included as a working-report page so the pair can annotate observations from the final execution environment before submission.

- Input assumptions verified.
- Source compiled successfully in the supplied environment.
- Functional output compared with the expected result.
- Machine-dependent observations identified separately from correctness evidence.
- Student A and Student B should initial this section after individual verification.

| Checkpoint | Status | Final observation |
|---|---|---|
| Design | Completed | _____ |
| Implementation | Completed | _____ |
| Execution | Completed | _____ |
| Analysis | Completed | _____ |
| Verification | Pending final signature | _____ |